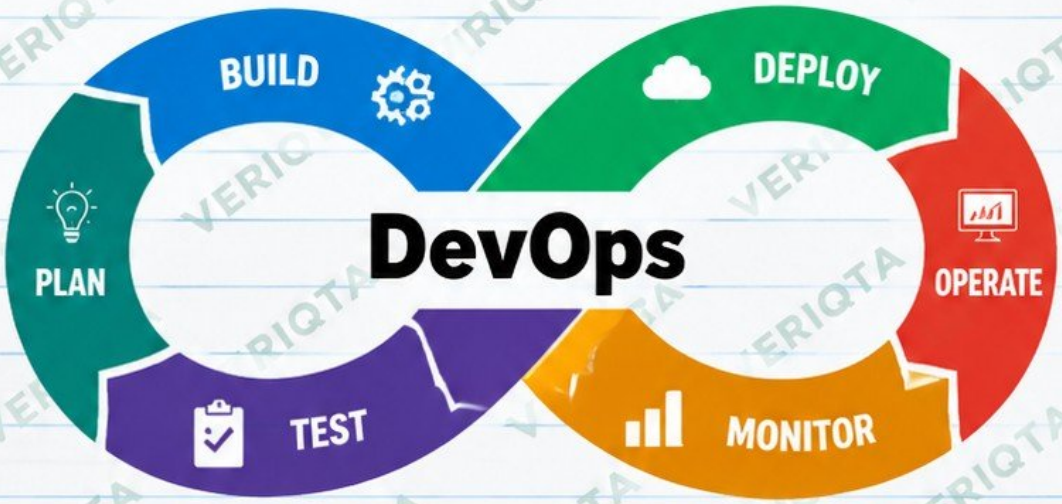
VERIQTA

The DevOps Glossary:

100 Terms Explained Simply





Linux | Git | CI/CD | Docker | Kubernetes | Cloud | Automation
Security | Monitoring | Reliability | and More

@veriqta @veriqta @veriqta @veriqta @veriqta @veriqta

Page 01. How Applications Run

1 Server

- A **server** is a computer or software system that provides services to other computers.
- It stores applications, files and data, and responds to requests from clients.
- **Example:** A web server that serves a website when someone visits it.

2 Client

- A **client** is an application or device that requests a service from a server.
- **Examples:** your web browser, mobile app, or another server.
- The client sends a request and receives a response.
- **Example:** Your browser requests a web page from a web server.

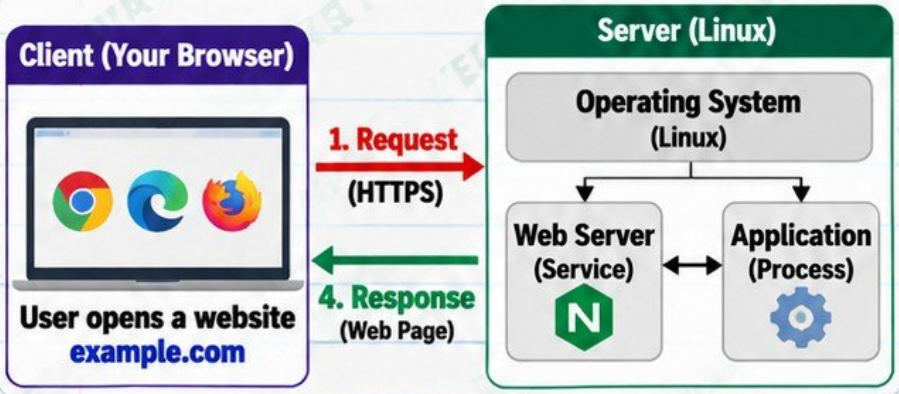
3 Operating System (OS)

- The **operating system (OS)** manages the computer's hardware and provides services for applications.
- It handles the CPU, memory, storage, networking, users and permissions.
- **Example:** Linux is a popular OS used on servers in DevOps.

4 Process

- A **process** is a running instance of a program.
- Each process uses system resources such as CPU and memory.
- **Example:** When a web server starts, it runs as a process.

How a Website Request Works



5 Service

- A **service** is a program or capability that provides a function to other programs or users, often running in the background.
- Services can automatically start, restart and keep running.
- **Example:** Nginx, Apache or a database service.
- Service = a long-running process that provides a specific function.

Step by Step: What Happens

1. The client (your browser) sends a request to the server.
2. The server's operating system receives the request.
3. A web server service (e.g. Nginx) running as a process handles the request.
4. The application process generates the response.
5. The server sends the response back to the client.

Key Takeaway

A website works because the client and server communicate, the operating system manages resources, a service (web server) runs as a process, and the application generates the content.

Real DevOps Connection

In real environments, you will manage Linux servers, configure services, monitor processes, and make sure applications respond to client requests.

Page 02. Navigating Linux

6 Terminal

- A **terminal** is a text-based interface used to interact with a computer.
- You type commands and the system gives you text output.
- **Example:** GNOME Terminal, macOS Terminal, or a remote SSH terminal.



7 Shell

- A **shell** is a program that interprets commands, such as **Bash**.
- It takes your command, understands it, and tells the operating system what to do.
- **Common shells:** Bash, Zsh, Fish.
- You are usually running a shell inside a terminal.
- **Example:** bash is the default shell on most Linux servers.

8 Command

- A **command** is an instruction given to a computer through a command interface.
- Commands can manage files, processes, users, networking and more.
- **Example:** ls, cd, pwd, mkdir.

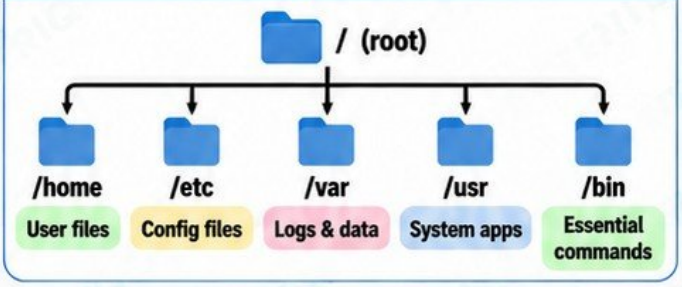
Common Linux Commands

pwd	Shows current directory
ls	Lists files and directories
cd /path	Change directory
mkdir folder	Create a directory
rm file	Remove a file

9 Filesystem

- The **filesystem** is the organized structure used to store and access files and directories on a Linux server.
- Everything starts from the root directory /.
- Important directories: /home, /etc, /var, /usr, /bin, /opt.
- It helps you know where files, applications and configurations are located.

Linux Filesystem Structure



10 Path

- A **path** is the location of a file or directory within the filesystem.
- **Absolute path:** starts from the root /
  Example: /home/ubuntu/projects
- **Relative path:** relative to your current location
  Example: ../projects or ../logs

Absolute vs Relative Path

Absolute Path (starts from /)	Relative Path (relative to current directory)
/home/ubuntu	./folder
/etc/nginx	../folder
/var/log	projects/app

Common Beginner Mistake

- Confusing the terminal with the shell.
  The terminal is the interface, the shell interprets your commands.
- Confusing relative and absolute paths.
  ./folder is different from /folder.

Practice This

1. Run pwd to see your current directory.
2. Run ls to list files and directories.
3. Run cd / to go to the root directory.
4. Run cd /home to explore a different directory.
5. Create a folder with mkdir test and verify with ls.

Page 03. Linux Access and Permissions

11 User

- A **user** is an identity used to access a system and run processes.
- Each user has a home directory and specific permissions.
- **Example:** **alice** is a normal user account.




```
# see current user
whoami → alice
```

12 Group

- A **group** is a collection of users used to manage shared access.
- Multiple users can belong to the same group.
- **Example:** developers group allows team members to access the same files.

```
# view your groups
groups
```



13 Permissions

- **Permissions** control who can **read (r)**, **write (w)** or **execute (x)** a file or directory.
- Each file has permissions for the owner, group and others.
- **Example:** **-rwxr-xr--** means owner can read, write, execute, group can read and execute, others can read.

Understanding Permission Format

-rwxr-xr--

Owner	Group	Others
r = read	r = read	r = read
w = write	- = no write	- = no write
x = execute	x = execute	- = no execute

Check File Permissions

```
user@server:~$ ls -l file.txt
-rwxr-xr-- 1 alice developers 1024 Oct 15 10:30 file.txt
```

File type (- = file) | Permissions (rwxr-xr--) | Owner (alice) | Group (developers) | Size (1024)

Common Beginner Mistake

- Using **chmod 777** to fix permission errors. This gives full access to everyone and is a security risk.
- Instead, give only the permissions needed.

```
# dangerous - do not use as a quick fix
chmod 777 file.txt
```

14 Root

- **Root** is the Linux superuser account with extensive administrative privileges.
- Root can install software, change system configuration, and manage all files.
- **Example:** The root user is used for system administration and maintenance.

```
# switch to root (if authorized)
sudo -i
```



15 sudo

- **sudo** runs an authorized command with another user's privileges, usually root.
- It allows you to perform administrative tasks without logging in as root.
- **Example:** Install a package using sudo.

```
# install a package
sudo apt update
```



Security Insight

- Never use **chmod 777** as a universal solution to permission errors.
- Give the minimum permissions required.
- Use sudo only when necessary.
- Keep the root account secure and do not use it for everyday work.

Key Takeaway

- Users identify who you are.
- Groups help manage access.
- Permissions control what you can do.
- Root has full control.
- **sudo** gives you temporary elevated access.

Page 04. How Computers Communicate

16 IP Address

- An **IP address** is a numerical address used to identify a network interface for communication.
- It helps computers find and send data to each other across networks.
- **Example:** 93.184.216.34 is an IPv4 address for a website server.

17 DNS (Domain Name System)

- DNS translates a domain name (like example.com) to an IP address.
- It helps applications find the correct server to connect to.
- **Example:** DNS looks up example.com and returns 93.184.216.34.



18 Port

- A **port** is a number used to identify a network communication endpoint on a server.
- Different services use different ports.
- **Example:** HTTP uses port 80, HTTPS uses port 443.

Common Ports	
80	HTTP
443	HTTPS
22	SSH
25	SMTP
53	DNS

19 Protocol

- A protocol is a set of **rules** that governs how data is formatted, sent and received between systems.
- It defines how computers communicate and understand each other.
- **Example:** HTTP, HTTPS, TCP, and UDP are common protocols.



20 HTTP/HTTPS

- **HTTP** (HyperText Transfer Protocol) is used to exchange web requests and responses.
- **HTTPS** is the secure version of HTTP. It uses TLS to encrypt data and verify the server. This protects the data from being read or modified by others.
- **Example:** When you visit a website, your browser uses HTTPS (port 443) to securely connect to the server.

Key Difference

HTTP	HTTPS
• Not encrypted	• Encrypted with TLS
• Uses port 80	• Uses port 443
• Data can be read by others	• Data is protected and authenticated

How a Browser Finds and Connects to a Website



Common Beginner Mistake

- Thinking a domain name is the same as an IP address
- Confusing port 80 (HTTP) with port 443 (HTTPS)
- Assuming a website is unsafe just because it uses HTTP locally
- Not understanding that DNS returns an IP address
- Thinking HTTPS is just HTTP with a different port

Practical Commands

nslookup example.com	Look up domain and see the IP address
dig example.com	Get DNS information
ping example.com	Check if the server is reachable
curl -I https://example.com	Check the server response headers
telnet example.com 443	Test if port 443 is open

Page 05. Network Traffic and Access

21 Localhost

- **Localhost** is a hostname that refers to the computer itself through the loopback interface.
- It usually resolves to **127.0.0.1** (IPv4) or **::1** (IPv6).
- It is used to test services on your own machine.

Example:
http://127.0.0.1:8080
http://localhost:8080



22 Firewall

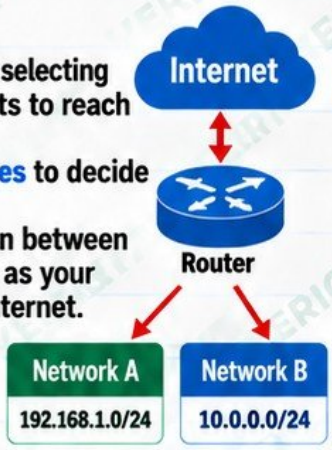
- A **firewall** is a security control that permits or blocks network traffic based on rules.
- It can be a software tool (e.g. UFW, iptables) or a cloud/network firewall (e.g. AWS Security Groups).
- It helps protect servers by allowing only required traffic.

Example: Allow only SSH (port 22)
sudo ufw allow 22



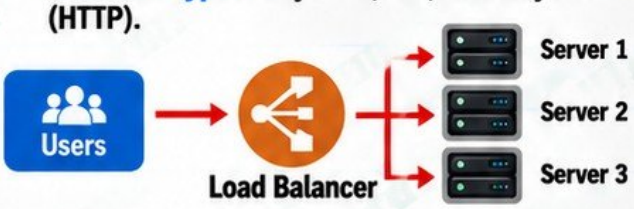
23 Routing

- **Routing** is the process of selecting paths for network packets to reach their destinations.
- Routers use **routing tables** to decide the best path.
- It enables communication between different networks, such as your home network and the internet.



24 Load Balancer

- A **load balancer** distributes incoming requests or connections across multiple backend servers.
- It helps improve availability, performance and fault tolerance.
- **Common types:** Layer 4 (TCP) and Layer 7 (HTTP).



25 SSH

- **SSH (Secure Shell)** is a protocol used for encrypted remote access and administration.
- It allows you to securely connect to a server, execute commands and manage files.
- It uses **port 22** by default.

Example: Connect to a server
ssh user@your-server-ip



Practical Example: Why 127.0.0.1 is Not Reachable

Application listening on localhost only

sudo python app.py
Listening on 127.0.0.1:8000



Server
127.0.0.1:8000
Only accessible from this server itself

Trying to access from another device



http://YOUR_SERVER_IP:8000
Connection refused

- The application is only listening on 127.0.0.1 (localhost).
- It is not listening on the server's public IP.
- You must configure it to listen on **0.0.0.0** and allow the port in the firewall.

Common Failures

- Application is running but unreachable.
- Service is listening on 127.0.0.1 instead of 0.0.0.0.
- Firewall is blocking the port.
- Wrong port is used.
- Routing or network rules are misconfigured.

How to Troubleshoot

1. Check if the service is running.
2. Check the listening address (127.0.0.1 or 0.0.0.0).
3. Verify the correct port.
4. Check firewall rules.
5. Test connectivity with a tool (curl, telnet, nc).

Key Takeaway

- Localhost (**127.0.0.1**) means only this machine.
- Use **0.0.0.0** to allow external access (when appropriate).
- A firewall controls which traffic is allowed.
- A load balancer distributes traffic.
- SSH provides secure remote access.

Page 06. Understanding Git

26 Version Control

- **Version control** is a system for tracking changes to files over time.
- It keeps a history of who changed what, when and why.
- It helps teams work together without losing or overwriting each other's work.
- **Example:** Git is a popular version control system.



Track changes over time

27 Git

- Git is a **distributed version** control system used to manage source code history.
- It runs on your local machine and also works with remote repositories.
- It tracks changes, supports branching, and makes collaboration easy.
- **Example:** Git is widely used in DevOps and software development.

28 Repository

- A **repository (repo)** is a collection of files and their version history.
- It stores all project files, commits, branches and metadata.
- Repositories can be **local** (on your computer) or remote (e.g. GitHub).
- **Example:** A web application's source code is stored in a Git repository.



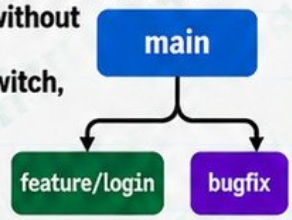
29 Commit

- A **commit** is a recorded **snapshot of** changes in Git history.
- Each commit has a unique ID, author, date and a message describing the changes.
- Commits help you track what was changed and why.
- **Example:** Save your work with a clear commit message.

```
git commit -m "Add login feature"
[main a1b2c3d] Add login feature
3 files changed, 45 insertions(+)
```

30 Branch

- A **branch** is a movable reference to a line of **development**.
- Branches allow you to work on new features or fixes without the main code.
- You can create, switch, merge and delete branches.
- **Example:** Create a new branch for a feature.




```
git switch -c feature/login
# Creates and switches to a new branch
```

Basic Git Workflow



Practical Example

1. Create a new repository: git init
2. Add a file: git add README.md
3. Commit the file: git commit -m "Initial commit"
4. Create and switch to new branch: git switch -c feature/setup
5. Make changes, add and commit.

Common Beginner Mistakes

- Thinking Git is the same as GitHub. (Git is a tool, GitHub is a hosting platform)
- Not committing changes regularly.
- Working directly on the main branch.
- Forgetting to pull before pushing.
- Using unclear commit messages.

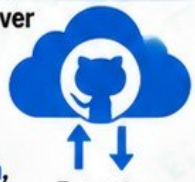
Key Takeaway

- Version control helps you track, collaborate and recover changes.
- Git manages your code history.
- Repositories store your project.
- Commits save changes.
- Branches let you work safely on new ideas.

Page 07. Team Collaboration with Git

31 Remote

- A **remote** is a named reference to another Git repository, commonly hosted on a server such as GitHub, GitLab or Bitbucket.
- It allows you to push and pull changes between your local repository and the remote repository.
- You can have multiple remotes (e.g. **origin**, upstream).
- **Example:** **origin** is the default remote name for your main repository.




```
git remote -v
origin  https://github.com/veriqta/app.git (fetch)
origin  https://github.com/veriqta/app.git (push)
```

32 Clone

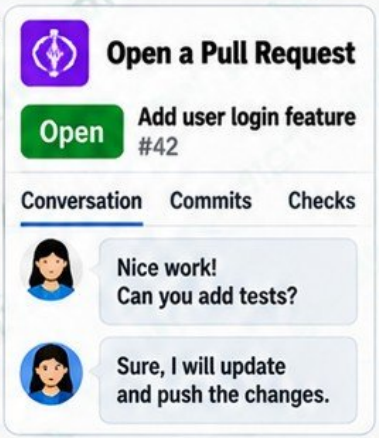
- **Clone** creates a **local copy** of a Git repository and its entire history from a remote server.
- It downloads all files, commits, branches and metadata to your computer.
- You can then work on the project locally.
- **Example:** Clone a repository from GitHub.




```
git clone https://github.com/veriqta/app.git
cd app
ls -la
```

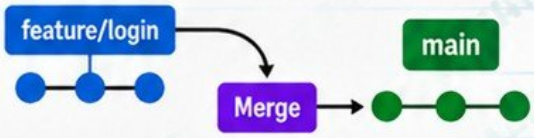
33 Pull Request

- A **pull request (PR)** is a proposal to review and merge changes into another branch, usually the **main** branch.
- It allows team members to review code, suggest changes and discuss before merging.
- Pull requests are commonly used on GitHub, GitLab and Bitbucket.
- **Example:** You create a PR to merge your feature branch into **main**.



34 Merge

- **Merge** combines changes from one branch into another branch.
- It brings the commit history and code changes together.
- After a successful merge, the changes from the feature branch become part of the target branch (e.g. **main**).
- **Example:** Merge a feature branch into **main** after review.



35 Merge Conflict

- A **merge conflict** happens when Git cannot automatically reconcile competing changes to the same part of a file.
- It occurs when two people (or branches) modify the same lines and Git is not sure which change to keep.
- You must manually review the file, resolve the conflicts and then complete the merge.
- **Example:** Git marks conflicting lines in the file and you edit the file to choose the correct version.

```
<<<<<<< HEAD
console.log("Login successful");
=======
console.log("User logged in");
>>>>>>> feature/login
```

Common Beginner Mistakes

- Not using pull requests for review.
- Merging directly into main.
- Ignoring merge conflicts.
- Overwriting other people's work.
- Not pulling the latest changes before pushing.

Team Workflow Example

1. Create a new branch: `git switch -c feature/login`
2. Make changes and commit: `git add . && git commit -m "Add login feature"`
3. Push to the remote: `git push -u origin feature/login`
4. Open a pull request on GitHub for review.
5. After approval, merge the pull request into main.

Key Takeaway

- Git enables multiple engineers to work on the same project safely.
- Use branches and pull requests to collaborate.
- Review changes before merging.
- Resolve conflicts instead of overwriting work.
- Keep your local repository up to date.

Page 08. CI/CD Fundamentals

36 Continuous Integration (CI)

- **Continuous Integration (CI)** means frequently integrating code changes into a shared repository.
- Each change is automatically built and validated through tests and checks.
- It helps catch errors early and keeps the codebase stable.
- **Example:** Every push to GitHub triggers automated tests.



37 Continuous Delivery

- **Continuous Delivery** keeps software in a tested, deployable state so releases can happen on demand.
- All code changes go through the full pipeline, including build, tests and security checks.
- The software is ready to be released manually when needed.
- **Example:** After passing all checks, the application is ready to deploy to production with one approval.



38 Continuous Deployment

- **Continuous Deployment** automatically releases changes that pass the required checks into production.
- It removes manual approval and deploys approved code to users automatically.
- It requires strong automated testing and monitoring.
- **Example:** Every successful pipeline run deploys the application to production automatically.



39 Pipeline

- A **pipeline** is an automated sequence of steps for building, testing and delivering software.
- It can include build, tests, security scans, container image build and deployment.
- Pipelines are defined in files (e.g. GitHub Actions, GitLab CI, Jenkins).
- **Example:** A CI/CD pipeline runs every time you push code to the repository.



40 Runner

- A **runner** is a machine or execution environment that performs pipeline jobs.
- It runs the commands defined in the pipeline file.
- Runners can be hosted by the platform (e.g. GitHub hosted runners) or self-hosted on your own servers.
- **Example:** A GitHub Actions runner downloads your code, runs tests and builds the application.



- Checkout code
- Run jobs
- Execute scripts
- Build and test
- Report results

How CI/CD Works After a Code Push

1. Code push to repository (e.g. GitHub)
2. Pipeline is triggered automatically
3. Code is built
4. Automated tests run (unit, integration, security checks)
5. Deployment (manual approval or automatic)
6. Application is updated in the target environment

Common Beginner Mistake

- Assuming Continuous Delivery and Continuous Deployment are the same.
- Thinking the pipeline only builds the code and does not run tests.
- Not setting up proper tests and security checks.
- Relying only on manual testing.
- Deploying directly to production without validation.

Key Difference

Continuous Delivery	Continuous Deployment
Fully tested and ready to deploy	Fully tested and ready to deploy
Manual release when needed	Automatic release to production
More control over when to release	Faster but requires strong testing and monitoring

Key Takeaway

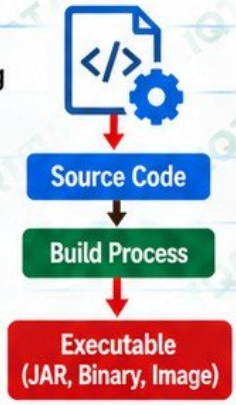
- CI/CD helps you deliver software faster, safer and with fewer errors.
- Pipelines automate the work from code to production.
- Runners execute the jobs.
- Continuous Delivery gives you control. Continuous Deployment automates the release.

Page 09. Building and Releasing Software

VERIQTA

41 Build

- **Build** is the process of transforming source code and dependencies into a usable software output.
- It compiles code, installs dependencies and creates a runnable package.
- The build process can produce **artifacts** such as a binary, JAR file or a container image.
- **Example:** Build a Java application using Maven to create a JAR file.



42 Artifact

- An **artifact** is a generated output from the build process.
- It can be a compiled application package, a JAR file, a ZIP file or a container image.
- Artifacts are stored in a registry or repository for later use.
- They are versioned so you can track and reproduce releases.
- **Example:** A Docker image stored in a container registry.



43 Automated Test

- An **automated Test** is a programmatic check that verifies expected software behavior.
- It runs automatically in the pipeline after the build.
- Tests can include unit tests, integration tests and security tests.
- They help catch errors early and prevent faulty code from reaching production.
- **Example:** Run unit tests using **pytest** or JUnit.



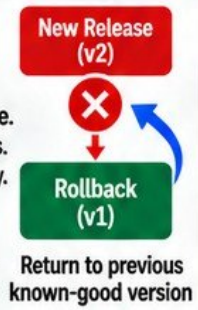
44 Deployment

- **Deployment** is the process of placing a software version into a target environment.
- It can be a staging environment for testing or a production environment for real users.
- Deployment can be done manually or automatically through a pipeline.
- It replaces the previous version with the new version.
- **Example:** Deploy a container image to a Kubernetes cluster.



45 Rollback

- **Rollback** means returning to a previous known-good application or infrastructure state.
- It is used when a new release causes problems.
- Rollbacks reduce downtime and restore stability.
- They can be automatic or manual, depending on your setup.
- **Example:** Roll back to the previous container image version.



End-to-End Software Release Flow



Production Insight

- A successful deployment command does not prove that an application works correctly.
- You must verify the application after deployment using health checks, logs and real user testing.
- Always monitor the application and be ready to roll back if needed.

Practical Example

1. Code is pushed to the repository.
2. Pipeline builds the application.
3. Automated tests run and must pass.
4. Artifact is published to a registry.
5. Application is deployed to staging or production.
6. Verify the release and monitor the application.

Key Takeaway

- A code change goes through build, test, artifact creation, deployment and verification.
- Artifacts are versioned for traceability.
- Rollbacks help you recover quickly from issues.
- Never assume a deployment is successful without verification.

Page 10. Docker Fundamentals

46 Container

- A **container** is an isolated process or group of processes packaged to run with defined dependencies and configuration.
- It includes the application, its runtime, libraries and settings.
- Containers share the host OS kernel but run in isolation from each other.
- They are lightweight, fast to start and portable across environments.
- **Example:** A web application running inside a Docker container.



Running Container (isolated)

47 Container Image

- A **container image** is a packaged filesystem and metadata used to create containers.
- It contains the application, runtime, libraries and configuration files.
- Images are read-only templates.
- A container is created from an image and runs as a writable instance.
- **Example:** An Nginx image from Docker Hub used to run a web server.



Container Image (read-only)

48 Dockerfile

- A **Dockerfile** is a text file containing instructions for building a container image.
- It defines the base image, copies files, installs dependencies and sets the startup command.
- It is used to create reproducible and consistent images.
- **Example:** A simple Dockerfile for a Python application.

```
# Dockerfile
FROM python:3.11-slim
WORKDIR /app
COPY . .
RUN pip install -r
 requirements.txt
CMD ["python", "app.py"]
```

49 Container Registry

- A **container registry** is a service that stores and distributes container images.
- It allows you to push images, share them and pull them on different environments.
- **Examples:** Docker Hub, Amazon ECR, GitHub Container Registry, GitLab Container Registry.
- **Example:** Push a built image to Docker Hub.



Container Registry (Docker Hub)

`docker push`
`docker pull`
`docker images`

50 Volume

- A **volume** is storage managed independently of a container's writable layer.
- It is commonly used to persist data such as databases, logs or uploads.
- Volumes remain available even if the container is deleted.
- **Example:** Use a volume to persist a database's data.



Docker Volume (persistent storage)

Docker Workflow: From Code to Running Container



1 Dockerfile	2 Build Image	3 Push to Registry	4 Run Container
Write a Dockerfile	`docker build -t myapp:1.0 .`	`docker push myapp:1.0`	`docker run -d -p 80:80 myapp:1.0`

Common Beginner Mistake

- Treating containers and container images as interchangeable.
- Thinking a container is the same as a Dockerfile.
- Not using volumes for data that needs to persist.
- Building images directly on production servers.
- Keeping unnecessary files in the image, making it too large.

Practical Example

1. Write a Dockerfile for a simple web app.
2. Build the image: `docker build -t veriqta-app .`
3. Run the container: `docker run -d -p 80:80 veriqta-app`
4. Verify it is running: `docker ps`
5. Push the image to a registry: `docker push veriqta-app`
6. Pull and run it on another server to confirm it works.

Key Takeaway

- Containers package everything your application needs.
- They run consistently across different environments.
- A Dockerfile defines how to build the image.
- Images are stored in a registry.
- Volumes are used to persist data.
- Containers and images are different but work together.

Page 11. Kubernetes Fundamentals

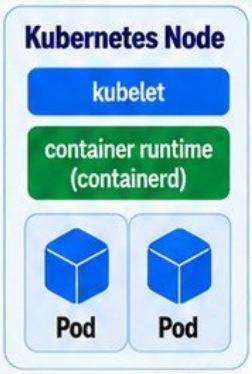
51 Cluster

- A **cluster** is a collection of machines and control components working together to run Kubernetes workloads.
- It includes a control plane that manages the cluster and worker nodes that run your applications.
- A cluster provides high availability, scalability and self-healing.
- **Example:** A production cluster running web applications across multiple nodes.



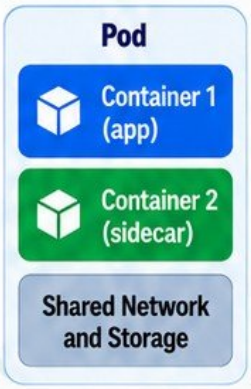
52 Node

- A **node** is a machine that participates in a Kubernetes cluster.
- It can be a physical server or a virtual machine.
- Nodes run the container runtime (e.g. containerd) and host the Pods.
- A cluster usually has multiple nodes for high availability.
- **Example:** A worker node running application Pods.



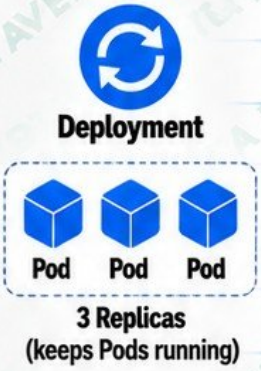
53 Pod

- A **pod** is the smallest deployable unit in Kubernetes.
- It can contain one or more containers that share the same network and storage.
- Each Pod gets its own IP address within the cluster.
- Pods are short-lived and can be recreated if they fail.
- **Example:** A Pod running a web application container.



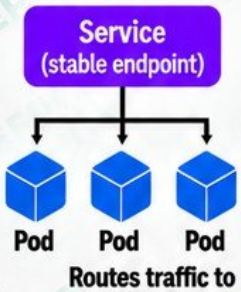
54 Deployment

- A **deployment** manages a set of replicated Pods.
- It keeps the desired number of Pods running.
- It automatically replaces failed Pods and supports rolling updates.
- You update the application by changing the container image in the Deployment.
- **Example:** A Deployment with 3 replicas of a web application.



55 Service

- A **service** provides a stable network endpoint to reach a changing set of backend Pods.
- It automatically routes traffic to healthy Pods.
- It can be an internal service (ClusterIP) or an external service (NodePort, LoadBalancer).
- It works with Deployments to ensure users can reach your application.
- **Example:** A Service that exposes a web application to other services or to the internet.



How Kubernetes Keeps Applications Running

1. You create a Deployment with the desired number of Pods.
2. Kubernetes schedules Pods on available nodes.
3. If a Pod fails, Kubernetes automatically creates a new one.
4. A Service provides a stable network endpoint to reach the Pods.
5. You update the Deployment with a new image.
6. Kubernetes performs a rolling update without downtime.



Common Beginner Mistake

- Thinking a Pod is the same as a Deployment.
- Confusing a container with a Pod.
- Assuming a Service runs the application (it only routes traffic).
- Expecting Pods to stay on the same node forever (they can be rescheduled).
- Not understanding the difference between ClusterIP, NodePort and LoadBalancer.

Practical Example

```
# Create a Deployment
kubectl create deployment veriqta-app \
  --image=nginx --replicas=3

# Expose it with a Service
kubectl expose deployment veriqta-app \
  --port=80 --target-port=80 --type=ClusterIP

# Check resources
kubectl get pods,svc
```

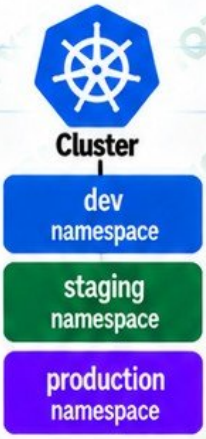
Key Takeaway

- Kubernetes keeps your containerized applications running and reachable.
- Deployments maintain the desired number of Pods.
- Services provide a stable network endpoint.
- Pods can be replaced, updated and rescheduled automatically.
- Together, they give you a reliable and scalable application platform.

Page 12. Kubernetes Configuration and Health

56 Namespace

- A namespace is a logical boundary used to organize many Kubernetes resources within a cluster.
- It helps separate resources for different teams, applications or environments.
- Each namespace has its own set of resources such as Pods, Deployments, Services, ConfigMaps and Secrets.
- **Example:** Create a namespace for a development environment.



57 ConfigMap

- A ConfigMap is a Kubernetes object used to store non-sensitive configuration data.
- It allows you to separate configuration from your container image.
- ConfigMaps can be used as environment variables or mounted as files.
- **Example:** Store application settings such as a database host or log level.

ConfigMap
(application configuration)

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  APP_ENV: production
  LOG_LEVEL: info
  DB_HOST: db-service
```

58 Secret

- A Secret is a Kubernetes object intended for sensitive data.
- It is used to store information such as passwords, API keys or tokens.
- Secrets should have appropriate access controls and be protected using best practices.
- They can be used as environment variables or mounted as files.
- **Example:** Store a database password for an application.

Secret
(sensitive data)

```
apiVersion: v1
kind: Secret
metadata:
  name: db-secret
type: Opaque
data:
  DB_PASSWORD:
   c2VjcmV0UGFzc3dvcmQ=
```

59 Liveness Probe

- A liveness probe is a check Kubernetes uses to decide whether a container should be restarted.
- If the probe fails, Kubernetes restarts the container.
- It helps recover from application hangs or deadlocks.
- **Examples:** HTTP request, TCP check or command execution.

Liveness Probe
(restart if unhealthy)

```
livenessProbe:
  httpGet:
    path: /health
    port: 8080
 initialDelaySeconds: 10
 periodSeconds: 10
 failureThreshold: 3
```

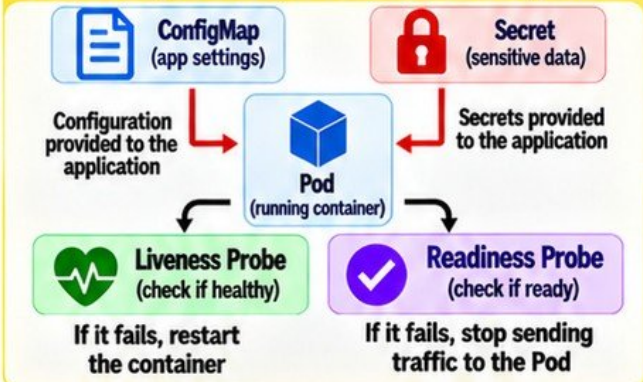
60 Readiness Probe

- A readiness probe is a check Kubernetes uses to determine whether a container is ready to receive traffic.
- If the probe fails, Kubernetes stops sending traffic to the Pod but does not restart the container.
- It ensures traffic only goes to Pods that are ready to serve requests.
- **Examples:** HTTP request, TCP check or command execution.

Readiness Probe
(ready to receive traffic)

```
readinessProbe:
  httpGet:
    path: /ready
    port: 8080
 initialDelaySeconds: 5
 periodSeconds: 10
 failureThreshold: 3
```

How Configuration and Health Work Together



Production Insight

- Incorrect health probes can create outages even when the application itself is functioning.
- A probe may be too strict, check the wrong path or depend on a slow external service.
- This can cause Kubernetes to restart healthy containess or stop sending traffic to working Pods.

Practical Example

1. Create a namespace for your application.
2. Store non-sensitive settings in a ConfigMap.
3. Store sensitive data in a Secret.
4. Deploy your application with liveness and readiness probes.
5. Verify that the Pod is running and ready:

```
kubectl get pods
kubectl describe pod <pod-name>
```

Key Takeaway

- Namespaces help organize resources in a cluster.
- ConfigMaps store non-sensitive configuration data.
- Secrets store sensitive data and must be protected.
- Liveness probes restart unhealthy containers.
- Readiness probes control when traffic is sent to a container.
- Correct configuration and health checks keep applications reliable and available.

Page 13. Cloud Computing Essentials

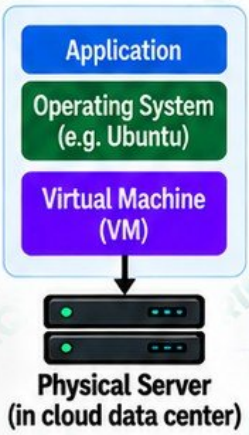
61 Cloud Computing

- **Cloud Computing** is on-demand access to computing resources delivered through a provider.
- It includes compute, storage, networking, databases and many other services.
- You pay for what you use instead of owning physical servers.
- It allows you to scale quickly, save time and focus on your applications.
- **Example:** Run a web application using a cloud provider such as AWS, Azure or Google Cloud.



62 Virtual Machine (VM)

- A **virtual machine (VM)** is a software-defined computer running its own operating system.
- It behaves like a physical server but runs on cloud infrastructure.
- You can choose the OS, CPU, memory and storage.
- Multiple VMs can run on the same physical hardware.
- It is useful for running applications, testing environments or legacy systems.
- **Example:** Launch an Ubuntu VM in a cloud provider and install a web server.



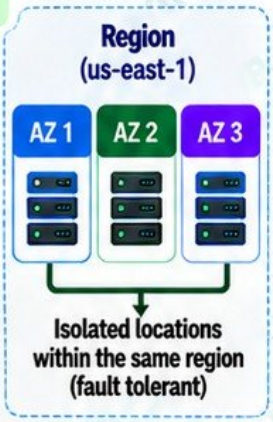
63 Region

- A **region** is a geographic area in which a cloud provider operates infrastructure.
- Each region contains multiple availability zones.
- Regions are located in different countries or cities around the world.
- You choose a region based on latency, data residency, compliance or user location.
- **Example:** aws eu-west-1 (Ireland), us-east-1 (N. Virginia).



64 Availability Zone

- An **availability zone (AZ)** is an isolated infrastructure location within a cloud region.
- Each AZ has its own power, networking and cooling.
- AZs are designed to be independent so that a failure in one AZ does not affect the others.
- You can run your application across multiple AZs for high availability and fault tolerance.
- **Example:** A web application running in two or more availability zones.

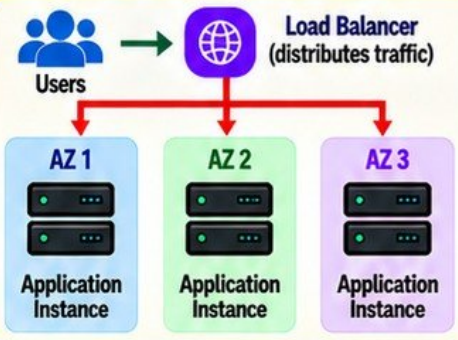


65 Scalability

- **Scalability** is a system's ability to handle changing demand by adjusting its resources.
- You can scale up by increasing the size of resources (e.g. larger VM).
- You can scale out by adding more instances (e.g. more servers).
- Cloud providers offer auto scaling to adjust resources automatically based on demand.
- **Example:** Automatically add more application instances when traffic increases.



Deploying Across Multiple Availability Zones



- Traffic is distributed across multiple AZs
- If one AZ fails, others continue to run
- Improves availability and fault tolerance
- Reduces dependence on one location

Key Difference

- On-premises: You buy, install, power and maintain physical servers.
- Cloud: You use provider infrastructure on demand without managing physical hardware.
- Cloud gives you flexibility, global reach, faster scaling and lower operational overhead.

Practical Example

1. Choose a cloud region (e.g. us-east-1).
2. Deploy your application across at least two availability zones.
3. Use a load balancer to distribute traffic.
4. Enable auto scaling to handle more users.
5. Test by simulating the failure of one AZ and confirm the application stays available.

Key Takeaway

- Cloud computing gives you on-demand access to infrastructure.
- A region is a geographic area, and it contains multiple availability zones.
- Availability zones are isolated locations for fault tolerance.
- Scalability helps you handle changing demand.
- Deploying across multiple availability zones reduces the risk of downtime.

Page 14. Cloud Networking and Storage

66 Virtual Private Cloud (VPC)

- A **Virtual Private Cloud (VPC)** is a logically isolated virtual network in a cloud environment.
- It gives you control over your IP address range, subnets, route tables and network gateways.
- You can launch resources such as virtual machines, databases and load balancers inside a VPC.
- It helps you create a secure and isolated environment for your applications.
- **Example:** Create a VPC with public and private subnets for a web application.

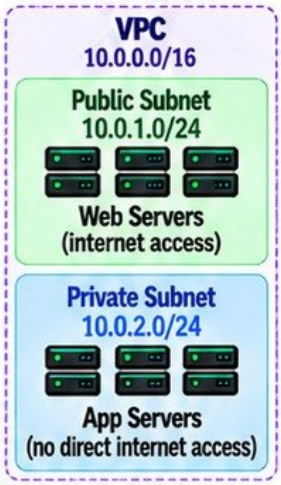


VPC



67 Subnet

- A **subnet** is a smaller IP network range within a larger network (such as a VPC).
- Subnets allow you to organize resources and control network access.
- You can create public subnets (for internet access) and private subnets (no direct internet access).
- Each subnet is in a specific Availability Zone.
- **Example:** A public subnet for web servers and a private subnet for application servers.



68 Object Storage

- **Object storage** stores data as objects, typically accessed through APIs.
- It is designed for high durability, scalability and cost-effectiveness.
- It is commonly used to store files such as images, videos, backups and application data.
- Data is stored in buckets and can be accessed from anywhere using APIs or SDKs.
- **Example:** Store user uploads, logs or backups in object storage.



Object Storage
(e.g. Amazon S3)

- Images
- Videos
- Backups
- Log files
- Application data

69 Block Storage

- **Block storage** is storage presented as addressable blocks, commonly used for virtual machine disks.
- It provides high performance and low latency.
- It can be attached to a VM like a physical disk.
- It is typically used for operating systems, applications and databases that need fast and reliable storage.
- **Example:** Attach a block storage volume to a virtual machine for a database server.



Block Storage
(e.g. EBS, Persistent Disk)

- High performance
- Low latency
- Attach to a VM
- Used for OS, applications and databases

70 Managed Database

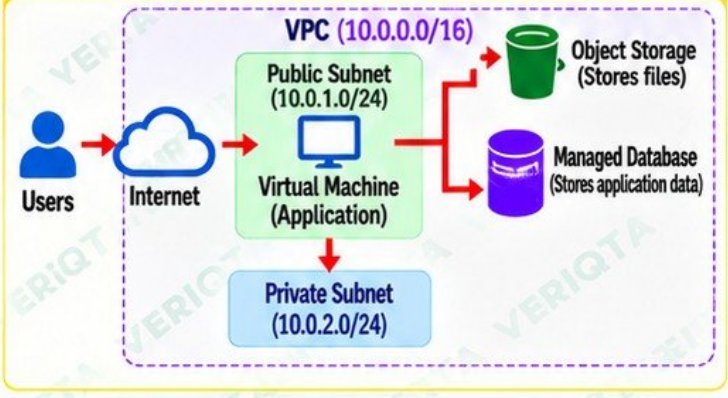
- A **managed database** is a database service where the provider handles selected operational responsibilities such as installation, patching, backups and scaling.
- It allows you to focus on your application instead of database infrastructure.
- Common examples include Amazon RDS, Azure Database and Google Cloud SQL.
- It provides high availability, backups and monitoring.
- **Example:** Use a managed database for a web application.



Managed Database
(e.g. RDS, Cloud SQL)

- Automatic Backups
- Patching & Updates
- High Availability
- Monitoring
- Scalability

How Cloud Networking and Storage Work Together



Key Difference

- Object storage stores data as objects (accessed via APIs).
- Block storage is a virtual disk for VMs (addressable blocks).
- Managed database is a database service with provider-managed operations.
- They serve different purposes and are not interchangeable.

Practical Example

1. Create a VPC with public and private subnets.
2. Launch a virtual machine in a subnet.
3. Store application files in object storage.
4. Use a managed database for application data.
5. Test that the application can access storage and database securely.

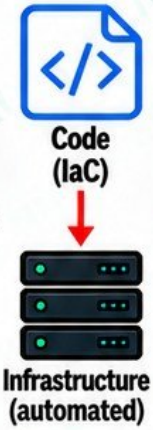
Key Takeaway

- A VPC provides a secure network environment.
- Subnets organize your resources and control access.
- Object storage is for files and unstructured data.
- Block storage is for virtual machine dissk and high performance needs.
- Managed databases reduce operational work and improve reliability.

Page 15. Infrastructure as Code

71 Infrastructure as Code (IaC)

- **Infrastructure as Code (IaC)** means managing infrastructure through version-controlled configuration rather than manual setup.
- It allows you to create, update and manage infrastructure using code.
- It makes infrastructure reproducible, consistent and easier to scale.
- It reduces human error and enables teams to work together.
- **Example:** Use Terraform to create a cloud server instead of setting it up manually in the cloud console.



72 Terraform

- **Terraform** is a tool used to declare and manage infrastructure through configuration files.
- It uses a simple, readable language called HashiCorp Configuration Language (HCL).
- It can create and manage resources across multiple cloud providers such as AWS, Azure and Google Cloud.
- It tracks the current state and compares it with your configuration to know what to change.
- **Example:** Use Terraform to create a virtual machine, network and storage in your cloud environment.

Terraform

Terraform Configuration (file: main.tf)

```
provider "aws" {
  region = "us-east-1"
}

resource "aws_instance"
"web" {
  ami = "ami-12345678"
  instance_type = "t2.micro"
}
```

73 Provider

- A **provider** is a Terraform plugin that enables interaction with a platform or service API.
- Each provider knows how to create, update and manage resources on a specific platform.
- **Examples:** AWS provider, Azure provider, Google Cloud provider.
- You specify the provider and configuration (such as region and credentials) in your Terraform files.

Common Terraform Providers

AWS Provider

Azure Provider

Google Cloud Provider

74 State

- **State** is Terraform's record of managed infrastructure objects and their associations with configuration.
- It maps real-world resources to your Terraform code.
- It helps Terraform know what exists, what has changed and what needs to be created, updated or deleted.
- The state file contains metadata and can include sensitive information.
- **Example:** Terraform stores state in a local file (terraform.tfstate) or a remote backend (e.g. S3).

Example: terraform.tfstate (simplified view)

```
{
  "resources": [
    {
      "type": "aws_instance",
      "name": "web",
      "id": "i-0abc123456",
      "attributes": {
        "instance_type": "t2.micro",
        "public_ip": "3.10.20.30"
      }
    }
  ]
}
```

75 Idempotency

- **Idempotency** means repeating an operation produces the same intended state without unintended additional changes.
- If the infrastructure already matches the configuration, Terraform makes no changes.
- This keeps your environment consistent and predictable.
- It helps avoid accidental duplicate resources or unnecessary updates.
- **Example:** Running terraform apply multiple times results in the same infrastructure state.

Idempotent Behavior

Run terraform apply → Infrastructure matches configuration → No changes are made

Terraform Workflow: From Code to Infrastructure

1. Write Configuration (main.tf)
2. Initialize Terraform
3. Preview Changes (plan)
4. Apply Changes (apply)
5. Infrastructure Created

```
# define resources
resource "aws_instance"
"web" {
 ami = "ami-123"
 instance_type = "t2.micro"
}
```

`terraform init` `terraform plan` `terraform apply`

Production Insight

- Terraform state may contain sensitive information such as resource IDs, IP addresses and credentials.
- Always protect your state file using a remote backend (e.g. AWS S3) with encryption and access controls.
- Do not store state files in public repositories or unprotected storage.

Practical Example

1. Write a Terraform configuration for a cloud server.
2. Run `terraform init` to initialize the provider.
3. Run `terraform plan` to preview the changes.
4. Review the plan output.
5. Run `terraform apply` to create the infrastructure.
6. Verify the server is created in your cloud environment.

Key Takeaway

- IaC allows you to manage infrastructure with code.
- Terraform helps you create and manage cloud resources.
- Providers enable Terraform to work with different platforms.
- State tracks your infrastructure and must be protected.
- Idempotency keeps your environment consistent.

Page 16. Monitoring and Observability

76 Monitoring

- **Monitoring** means collecting and evaluating system information to detect conditions requiring attention.
- It helps you understand the health, performance and availability of your applications and infrastructure.
- It uses metrics, logs and traces to give you visibility.
- It allows engineers to detect problems early and respond before users are affected.
- **Example:** Monitor a web application to ensure it is running, performant and available.



Availability
Performance
Errors
Resource Usage

77 Metric

- A **metric** is a numerical measurement of system behavior over time.
- Metrics help you track performance, resource usage and application behavior.
- They are usually collected at regular intervals (e.g. every 15 seconds).
- Common examples include CPU usage, memory usage, request count and response time.
- **Example:** Track the average response time of your application over the last 5 minutes.

Example Metrics



78 Log

- A **log** is a recorded event or message produced by a system or application.
- Logs provide detailed information about what happened, when it happened and sometimes why it happened.
- They are essential for debugging, troubleshooting and understanding application behavior.
- Logs can be collected from applications, servers, containers and cloud services.
- **Example:** A log entry showing an error when a user tries to log in.

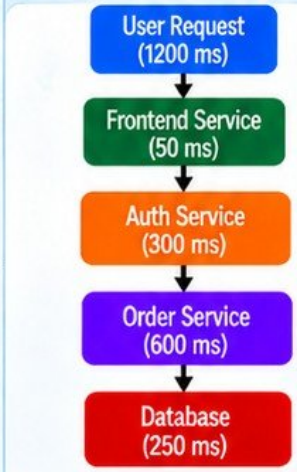
Example Log Entry

```
2026-10-15 10:24:03 INFO [auth] User login attempt email= user@example.com
2026-10-15 10:24:05 ERROR [auth] Login failed error= "Invalid credentials"
2026-10-15 10:24:06 INFO [auth] User locked reason= "Too many failed attempts"
```

79 Trace

- A **trace** is a record showing how a request moves through operations or services.
- Traces help you understand the flow of a request across multiple components in a distributed system.
- They show the time taken at each step and help identify where delays occur.
- Traces are especially useful in microservices architectures.
- **Example:** Trace a user request from the frontend through multiple services to find a slow operation.

Example Trace (Request Flow)



80 Alert

- An **alert** is a **notification** triggered when defined conditions indicate a potential problem.
- Alerts help engineers respond quickly to issues before users are affected.
- Alerts are usually based on metrics, logs or application health checks.
- Good alerts are specific, actionable and not too noisy.
- **Example:** Alert when the average response time is above 500 ms for 5 minutes.

Example Alert

```
Alert: High Latency
Service: web-app
Condition: Average response time > 500 ms
Duration: 5 minutes
Severity: High
Time: 2026-10-15 10:30:00
```

How Monitoring, Metrics, Logs and Traces Work Together

1. **Alert Triggered** — High latency triggers an alert.
2. **Check Metrics** — Metrics show the trend and confirm the issue.
3. **Analyze Traces** — Traces identify the slow operation.
4. **Review Logs** — Logs provide additional evidence.

Production Insight

- Without proper monitoring and observability, issues can go unnoticed until users are affected.
- Too many alerts can cause alert fatigue and lead to missed critical issues.
- Logs may contain sensitive information and should be handled securely.
- Traces and metrics together give a more complete picture than either one alone.

Practical Example

1. Set up monitoring for your application (e.g. CPU, memory, response time).
2. Create an alert for high latency.
3. Generate load and observe the metrics.
4. Use traces to identify the slow operation.
5. Check the logs for additional details.

Key Takeaway

- Monitoring helps you understand your application's health.
- Metrics show numerical data over time.
- Logs provide detailed event information.
- Traces show the flow of requests across services.
- Alerts notify you when something needs attention.

Page 17. Reliability and Incident Response

81 Site Reliability Engineering (SRE)

- **Site Reliability Engineering (SRE)** means applying software engineering approaches to improve service reliability and operations.
- It combines software engineering and operations to build reliable, scalable and maintainable systems.
- It uses automation, monitoring, measurement and continuous improvement.
- It focuses on preventing problems and reducing the impact of failures.
- **Example:** Using automation and monitoring to reduce downtime and improve user experience.



82 Incident

- An **incident** is an event that disrupts or threatens normal service operation.
- It can be caused by a failure, configuration change, increased demand or an external issue.
- Incidents can lead to partial or complete loss of service.
- They require a coordinated response to investigate, mitigate and restore service.
- **Example:** A web application returns errors for users due to a database connection failure.

Incident Example

Users see errors when accessing the application.

Detect issue (monitoring/alerts)

↓

Investigate (find root cause)

↓

Mitigate (restore service)

↓

Confirm recovery (monitor and validate)

83 Service Level Indicator (SLI)

- A **Service Level Indicator (SLI)** is a quantitative measure of a service's behavior.
- It shows how the service is performing from the user's perspective.
- Common SLIs include request success rate, latency and availability.
- SLIs are measured over a defined time period.
- **Example:** 99.5% of HTTP requests return a successful response (2xx) in the last 7 days.

Common SLIs

Request Success Rate
Percentage of successful requests (2xx).

Latency
Time taken to respond to a request.

Availability
Percentage of time the service is reachable.

84 Service Level Objective (SLO)

- A **Service Level Objective (SLO)** is a target value or range for an SLI over a defined period.
- It represents the reliability goal for a service.
- SLOs help teams set clear expectations and measure whether the service is meeting its goals.
- If the SLI falls below the SLO, the service is considered to be underperforming.
- **Example:** 99.9% request success rate over a 30-day period.

SLI vs SLO

SLI (measurement)
What is the current performance?
Example: 99.7% request success rate

SLO (target)
What is the goal?
Example: 99.9% request success rate for 30 days

85 Mean Time to Restore (MTTR)

- **Mean Time to Restore (MTTR)** is the average time required to restore service after a failure.
- It is measured from the time the incident is detected until the service is fully restored.
- A lower MTTR indicates a faster recovery and better operational readiness.
- MTTR helps teams understand and improve their incident response process.
- **Example:** If a service is down for 15 minutes and then restored, the MTTR for that incident is 15 minutes.

How MTTR is Measured

Incident Detected → Service Restored

MTTR = Time to Restore (e.g. 15 minutes)

Incident Response Flow (Example)

1. **Detect** — Identify the issue through monitoring or user reports.
2. **Investigate** — Find the root cause using metrics, logs and traces.
3. **Mitigate** — Apply a fix or workaround to restore service.
4. **Validate** — Confirm the service is working correctly.
5. **Learn** — Review the incident and implement improvements.

Production Insight

- Incident response should prioritize restoring service and preserving evidence over assigning blame.
- Keep clear communication with stakeholders during an incident.
- Collect and retain relevant logs, metrics and changes for analysis.
- Use the incident to learn and prevent similar issues in the future.

Practical Example

1. A monitoring alert shows a high error rate.
2. Investigate using metrics, logs and traces.
3. Identify a failing component (e.g. database).
4. Apply a fix and restore service.
5. Measure the MTTR and confirm the service is healthy.

Key Takeaway

- SRE uses software engineering to improve reliability.
- SLIs measure service behavior.
- SLOs set the reliability target.
- MTTR measures recovery time after a failure.
- When incidents happen, focus on restoring service, communicating clearly and learning from the incident.

Page 18. DevOps Security Essentials

86 Authentication

- **Authentication** means verifying the identity of a user, service, or system.
- It confirms who you are before granting access.
- Authentication can use passwords, tokens, biometric methods or certificates.
- It is the first step in the security process.
- **Example:** A user provides a username and password, or a CI/CD pipeline uses a token to authenticate with a cloud provider.

Authentication Methods

- Username and Password
- API Token
- Certificate
- Biometric (MFA)

87 Authorization

- **Authorization** means determining what an authenticated identity is allowed to do.
- It controls access to resources and actions.
- It is based on permissions, roles or policies.
- Authorization ensures that users and services can only perform allowed actions.
- **Example:** An authenticated user may be allowed to read a file but not delete it.

Authorization Example

User (authenticated)

- Read: Allowed
- Write: Allowed
- Delete: Denied
- Admin: Denied

88 Identity and Access Management (IAM)

- **Identity and Access Management (IAM)** means managing identities, roles and permissions across systems.
- It helps you control who can access what resources.
- IAM is used in cloud providers such as AWS, Azure and Google Cloud.
- It supports users, groups, roles and policies.
- **Example:** Create an IAM role for a CI/CD pipeline with limited permissions to deploy an application.

IAM Components

- Users (individual people)
- Groups (teams)
- Roles (temporary access)
- Policies (permissions)

89 Least Privilege

- **Least Privilege** means granting only the permissions necessary to perform an authorized task.
- It reduces the risk of accidental or malicious changes.
- It limits the impact if a credential is compromised.
- It should be applied to users, services, applications and infrastructure.
- **Example:** A deployment role can deploy to a specific environment but cannot delete infrastructure.

Principle of Least Privilege

Too Many Permissions (increases risk)

- Full admin access
- Can delete resources
- Can change all settings

Least Privilege (reduces risk)

- Only required permissions
- Limited to specific resources
- Cannot delete critical resources

90 Credential

- A **credential** is information or material used to establish or prove identity.
- **Examples** include a password, token, API key or private key.
- Credentials allow users or services to authenticate to systems and APIs.
- They must be protected and stored securely.
- **Example:** Use a short-lived token instead of a long-lived password in a CI/CD pipeline.

Common Credentials

- Password (user login)
- Token (API token)
- API Key (service access)
- Private Key (certificate)

CI/CD Pipeline Security Example

1. Source Code (in repository)
2. CI/CD Pipeline (authenticate with limited role)
3. Cloud Provider (deploy application with least privilege)
4. Application (deployed securely)

Do not store long-lived credentials in source code.
Use temporary credentials and least privilege permissions.

Security Insight

- Authentication proves who you are.
- Authorization determines what you may do.
- Always use least privilege.
- Protect credentials and use short-lived tokens where possible.
- Review and remove unnecessary access regularly.

Practical Example

1. Create an IAM role with limited permissions.
2. Use the role in your CI/CD pipeline.
3. Authenticate using a short-lived token.
4. Deploy the application.
5. Verify that the pipeline cannot perform unauthorized actions.

Key Takeaway

- Authentication verifies identity.
- Authorization controls what you can do.
- IAM manages users, roles and permissions.
- Use least privilege to reduce risk.
- Protect credentials and avoid embedding long-lived secrets in source code.

Page 19. Deployments and Production Safety

91 Environment

- An environment is a separate context where software runs, such as development, staging or production.
- Each environment has its own configuration, data and purpose.
- Environments help you test changes before they reach real users.
- They reduce risk and prevent accidental changes in production.
- Example: Develop new features in development, test them in staging and deploy to production when ready.

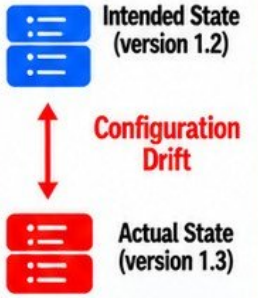
Common Environments

- **Development (Dev)** — Build and test new features
- **Staging** — Test before production
- **Production** — Live environment for real users

92 Configuration Drift

- Configuration drift refers to differences between the intended configuration and the actual state of a system.
- It happens when manual changes are made outside of version control.
- It can cause unexpected behavior, failures and security issues.
- Tools like Infrastructure as Code help prevent drift.
- Example: A server should have version 1.2 installed, but someone manually installs version 1.3, creating a drift from the intended configuration.

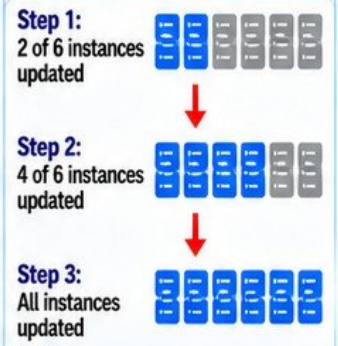
Configuration Drift Example



93 Rolling Deployment

- A rolling deployment updates application instances gradually instead of replacing all instances at once.
- It keeps the application available during the update.
- New versions are deployed to a subset of instances, verified, then rolled out to the rest.
- If issues are found, the rollout can be paused or rolled back.
- Example: Update 2 out of 6 instances first, verify health, then continue with the remaining instances.

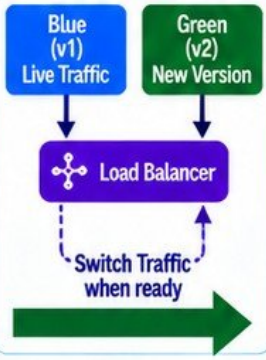
Rolling Deployment (Step by Step)



94 Blue-Green Deployment

- A blue-green deployment uses two identical environments (blue and green).
- One environment (blue) serves live traffic while the new version is deployed to the other environment (green).
- Once the new version is tested and verified, traffic is switched to the green environment.
- If there is an issue, traffic can be switched back to blue.
- Example: Deploy version 2 to green, test it, then switch traffic from blue to green.

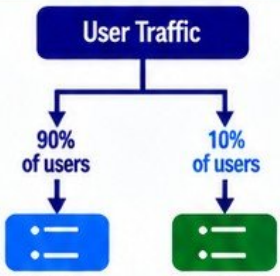
Blue-Green Deployment



95 Canary Deployment

- A canary deployment releases a change to a limited portion of users or traffic.
- It allows you to test the new version with real traffic and monitor for issues.
- If the new version is stable, traffic is gradually increased.
- If problems are detected, the change can be stopped or rolled back quickly.
- Example: Send 10% of traffic to version 2, monitor error rates, then increase to 50% and finally 100%.

Canary Deployment (Traffic Flow)



Why Deployment Strategies Matter

- Reduce downtime and user impact.
- Allow safe testing in real environments.
- Help detect issues early.
- Provide a way to roll back if something goes wrong.

Example Deployment Flow



Beginner Mistake

- Assuming every deployment strategy automatically provides zero downtime.
- Not testing the new version before sending full traffic.
- Skipping monitoring after deployment.
- Not having a rollback plan.

Practical Example

1. Deploy a new application version to a small number of instances or users.
2. Check application health, logs and error rates.
3. Gradually increase traffic if everything is stable.
4. Stop or roll back if issues are detected.
5. Document the deployment and lessons learned.

Key Takeaway

- Do not treat production deployment as simply copying files to a server.
- Use environments, testing and controlled rollout strategies.
- Always monitor and have a rollback plan.
- Deployment strategies reduce risk but do not automatically guarantee zero downtime.

Page 20. Debugging and Operating Real Systems

96 Troubleshooting

- **Troubleshooting** is a structured process for identifying and resolving technical problems.
- It involves gathering information, forming a hypothesis, testing, and applying a fix.
- It uses logs, metrics, system behavior and configuration data.
- It helps you understand what is happening instead of guessing.
- **Example:** An application is slow, you check metrics, logs and resource usage to find the cause and fix it.

Troubleshooting Steps

1. Identify the problem (what is happening?)
2. Gather information (logs, metrics, config)
3. Analyze and test (find the cause)
4. Apply a fix and verify

97 Root Cause Analysis (RCA)

- **Root Cause Analysis (RCA)** means investigating the contributing conditions and causes behind a failure.
- It goes beyond symptoms to find the real reason.
- It helps prevent the same issue from happening again.
- It looks at technical, process and people factors.
- **Example:** A service crashed due to high database connections. RCA finds the application did not release connections properly under load.

RCA Process

1. Understand the incident
2. Collect evidence (logs, metrics, events)
3. Identify contributing factors
4. Find the root cause and prevent recurrence

98 Runbook

- A **runbook** is documented instructions for carrying out an operational task or responding to a known situation.
- It provides clear, step-by-step actions.
- It helps teams respond faster and consistently.
- It reduces mistakes during incidents.
- **Example:** A runbook for high CPU usage includes checks, commands and steps to resolve the issue.

Example Runbook (High CPU)

1. Check the current CPU usage.
2. Identify the top processes.
3. Find the cause (application, system or external).
4. Apply the fix (restart, scale, or adjust configuration).
5. Verify the system is healthy.

99 Backup

- A **backup** is a recoverable copy of data or system information created to protect against loss.
- Backups help you recover from accidental deletion, configuration changes, corruption or system failure.
- They should be automated, regular and tested.
- **Example:** A database is backed up daily so it can be restored if it becomes corrupted or deleted.

Backup Flow

Application / Database → Create Backup (automated) → Store in Separate Location → Test Restore regularly

100 Disaster Recovery

- **Disaster Recovery (DR)** means the processes and capabilities used to restore critical systems after major disruption.
- It prepares for events like hardware failure, natural disasters or cloud outages.
- It includes backups, infrastructure, runbooks and testing.
- It aims to reduce downtime and data loss.
- **Example:** If a cloud region is unavailable, systems are restored in another region using backups and DR procedures.

Disaster Recovery Overview

Major Disruption (e.g. region outage) → Restore Systems (from backups) → Switch to DR Environment (another region) → Verify and Return to Service

Production Failure Scenario (Bringing It All Together)

1. Application Unavailable after Deployment
2. Investigate (metrics, logs, system behavior)
3. Find Failing Component (root cause)
4. Restore Service (apply fix or rollback)
5. Document Incident (update runbook)
6. Improve Recovery Procedures (prevent recurrence)

Example: Web Application Incident

After deploying a new version, users see errors. The team checks metrics and logs, finds a database connection issue, rolls back to the previous version, restores service, documents the incident and updates the runbook to prevent it from happening again.

Beginner Mistake

- Assuming logs always show the exact cause.
- Ignoring runbooks and troubleshooting without a plan.
- Skipping backups or not testing restores.
- Thinking disaster recovery is only for large companies.
- Assuming every issue can be fixed by restarting a service.

Final Junior Engineering Lesson

1. The goal is not to memorize 100 terms.
2. Understand how systems work together.
3. Recognize failures using evidence.
4. Use a structured approach to troubleshoot.
5. Make safe and informed engineering decisions.

Key Takeaway

- Use logs, metrics and runbooks to understand and fix issues.
- Find the root cause, not just the symptom.
- Keep regular backups and test them.
- Be prepared for major failures with disaster recovery.
- Continuous learning and improvement make systems more reliable.